

HandControlPC

Contrôle gestuel du PC par webcam

Documentation technique — Projet POO NFP101

Auteur : Mohamed Amine Sobhi

IA utilisée : Claude Code (Anthropic) — voir section 5 pour les détails.

1. Contexte et besoin

Les interfaces homme-machine traditionnelles reposent sur le clavier et la souris. HandControlPC propose une alternative : contrôler son PC via des gestes de la main captés par la webcam, sans matériel supplémentaire.

Cas d'usage :

- Contrôle du volume audio sans toucher le clavier
- Prise de capture d'écran sans raccourci clavier
- Zoom sur le flux vidéo via geste bimanuel
- Reconnaissance de signes (A, B, L) pour accessibilité

2. Fonctionnalités

Geste	Action	Retour au menu
Index seul levé	Mode Volume : pincement pouce-index	Main ouverte (5 doigts)
3 doigts levés	Mode Screenshot : maintenir ~0.7s	Automatique après capture
2 mains visibles	Mode Zoom : écarter/rapprocher les index	Une seule main restante

2.1 Reconnaissance de signes

Le module `test_sign_recognition.py` implémente la reconnaissance statique de 3 signes de la main (A, B, L) avec stabilisation temporelle (`HOLD_FRAMES = 20 frames`) et retour vocal (commande `say` de macOS).

3. Architecture et choix techniques

3.1 Structure des modules

Le projet est organisé en modules indépendants :

Fichier	Rôle
<code>main.py</code>	Point d'entrée — boucle principale, gestion des modes
<code>utils/gesture_action.py</code>	Classe abstraite <code>GestureAction</code> (ABC) — interface commune
<code>utils/hand_tracking.py</code>	<code>HandDetector</code> — détection MediaPipe et calcul <code>fingers_up</code>
<code>utils/volume_control.py</code>	<code>VolumeController</code> — hérite de <code>GestureAction</code>
<code>utils/screenshot.py</code>	<code>ScreenshotTaker</code> — hérite de <code>GestureAction</code>
<code>utils/zoom_control.py</code>	<code>ZoomManager</code> — hérite de <code>GestureAction</code>
<code>utils/config_loader.py</code>	Chargement JSON avec <code>deepcopy</code> et fallback
<code>config.json</code>	Configuration externe modifiable sans toucher au code
<code>tests/</code>	22 tests unitaires automatisés avec <code>pytest</code>

3.2 Héritage et polymorphisme

- La classe abstraite `GestureAction` définit le contrat commun à toutes les actions :
- Méthode abstraite `process(img, hands_data)` → implémentée par chaque sous-classe
 - Méthodes concrètes `activate()`, `deactivate()`, `render_hud()` — partagées par héritage
 - Polymorphisme : `main.py` appelle `process()` sans connaître le type exact de l'action

Hiérarchie :

```
GestureAction (ABC) ■■■■ VolumeController ■■■■ ScreenshotTaker ■■■■
ZoomManager
```

3.3 Bibliothèques choisies

Bibliothèque	Justification
OpenCV	Capture webcam, traitement d'image, affichage HUD
MediaPipe	Détection 21 landmarks de la main en temps réel
NumPy	Interpolation linéaire (volume), clipping (zoom)
MSS	Capture d'écran multi-plateforme
osascript	Contrôle volume et son macOS natif
pytest	Framework de tests unitaires
reportlab	Génération de la documentation PDF

4. Tests unitaires

- 22 tests automatisés dans `tests/` — tous exécutables sans webcam ni matériel :
- Reconnaissance de signes (A, B, L, inconnus)
 - Interpolation volume (distance → pourcentage)
 - Calcul de distance euclidienne
 - Reset et clipping du `ZoomManager`
 - Vérification de l'héritage (`isinstance GestureAction`)
 - Chargement config JSON, fallback sur valeurs par défaut, JSON invalide
 - Création du dossier `screenshots`
 - Comportement de `fingers_up` avec données vides

Commande d'exécution :

```
python -m pytest tests/ -v
```

5. Utilisation de l'IA

L'IA utilisée est Claude Code (Anthropic — `claude-sonnet-4-6`). Tout le code généré est encadré dans le source avec les marqueurs requis :

```
# ##### CODE IA (Claude) #####
```

Ce qui a été généré/aidé par IA	Prompt / Description	Ce que j'ai modifié/validé
Classe abstraite <code>GestureAction</code>	"Crée une classe de base ABC en Python pour adapter les signaux, vérifie <code>process()</code> utilisable et <code>render_hud()</code>	Adaptation des signaux, vérification <code>process()</code> utilisable et <code>render_hud()</code>
Refactorisation <code>VolumeController</code> , <code>ScreenshotTaker</code> , <code>ZoomManager</code>	"Fais hériter ces classes de <code>GestureAction</code> en implémentant la logique métier (calcul distance, capture) reste inchangé	Vérification que la logique métier (calcul distance, capture) reste inchangé
Module <code>config_loader.py</code>	"Crée un loader JSON avec <code>deepcopy</code> et fallback en cas d'erreur sur <code>shallow copy</code> (merged[section].update)	Identification et remplacement de <code>shallow copy</code> (merged[section].update)

Suite de tests pytest (22 tests)	"Génère des tests unitaires sans webcam pour figurer les cas d'usage et signifier les tests nécessaires"
Documentation PDF (ce fichier)	"Génère un script reportlab pour la documentation et génère le contenu du texte, fonctionnalités, limites)

Taux estimé d'utilisation IA : ~35% (architecture et tests). La logique de détection gestuelle, la boucle principale, et l'intégration MediaPipe/OpenCV ont été écrits et maîtrisés par l'auteur.

6. Limites et pistes d'amélioration

Limites actuelles :

- Limité à macOS (osascript, afplay) — portage Windows/Linux nécessite des alternatives
- Sensible à la luminosité et aux arrière-plans complexes
- Reconnaissance de signes limitée à 3 lettres (A, B, L)
- Pas de contrôle de la souris ou des fenêtres (prévu comme amélioration)

Pistes d'amélioration :

- Ajout du déplacement de souris par geste (index pointant)
- Support Windows/Linux via des APIs cross-platform
- Ajout d'un mode apprentissage pour enregistrer de nouveaux gestes
- Interface de configuration graphique (tkinter ou web)
- Système de logs persistants dans un fichier